Intro to Programming

A. A — Concept Application

Q.1 Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose Section the wrong methodology for this context.

Ans.

I would recommend Agile methodology for this project.

The reason is that the project requirements are frequently changing and the client wants to review working features every two weeks.

Agile allows the team to develop the project in small iterations and take regular feedback from the client.

If the wrong methodology is chosen, the team may face these two risks:

1.Changing requirements**:** Changes may cause extra work, cost and delay.

2. Delayed feedback**:** Problems may be found late, making them difficult and expensive to fix.

Q 2. Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Ans.

Our team should use a branch-based Git workflow.

Team members should not directly push incomplete code to the main branch.

Each member should create a separate branch, complete and test the code, and then send it for review.

After approval, the code should be merged into the main branch.

Git Commands:

1. git checkout -b feature-name – Creates a new branch for working on a feature without affecting the main branch.

2. git commit -m "message" – Saves the changes with a meaningful message.

Q.3 List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

Ans.

he appropriate HTML input types are:

- Applicant Name → text
- Email → email
- Phone Number → tel
- Preferred Domain → radio
- Resume → file

We use the required attribute to make all fields compulsory. The email input type checks the email format.

HTML Program:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internship Application</title>
</head>
<body>
```

```
<h2>Internship Application Form</h2>

<form>

Name:
<input type="text" required>
<br><br>

Email:
<input type="email" required>
<br><br>

Phone:
<input type="tel" required>
<br><br>

Preferred Domain:
<input type="radio" name="domain" required> Web
<input type="radio" name="domain"> Data
<input type="radio" name="domain"> Mobile
<input type="radio" name="domain"> AI
<br><br>

Resume:
<input type="file" required>
<br><br>

<input type="submit" value="Apply">
```

```
</form>


</body>
</html>
```

Q 4. Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Ans.

Bootstrap Grid System is used to create a responsive layout. It divides a row into 12 columns.

For the article column, we can use:

```
<div class="col-12 col-md-8 col-lg-9">
  Article Content
</div>
```

Explanation:

- col-12 → Full width on small/mobile screens.
- col-md-8 → Takes 8 columns on medium screens.
- col-lg-9 → Takes 9 columns on large screens.

Q 5. Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Ans.

An array is better because it can store the data of 30 days in one variable.

Using 30 separate variables would make the program difficult to manage.

The program works in two steps:

1.Calculate Average: Add all 30 days and divide by 30.

2.Find Above-Average Days: Compare each day's value with the average and display the days that are greater than the average.

Both steps cannot be done in one loop because we need to know the average first before comparing the daily values.

Q 6 Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Ans.

Step 1 – Empty String:
When the user enters nothing, the string is empty.

Step 2 – Why does it crash?
The program may crash because it tries to access a character without checking whether the string contains a character.

Step 3 – Pointer Check:
Before dereferencing the pointer, check that the string is not empty:

*p != '\0'

Step 4 – Pointer Traversal:

With a pointer, we use *p to access a character and p++ to move to the next character

.

Step 5 – Array Traversal:
With an array, we use str[i] to access each character.

Step 6 – Easier Method:
Array indexing makes the empty-string problem easier to catch because we can check:

str[0] == '\0'

B. B — Practical Coding Tasks

Q.1 Grade Band Checker

Accept a float percentage value as user input using scanf().

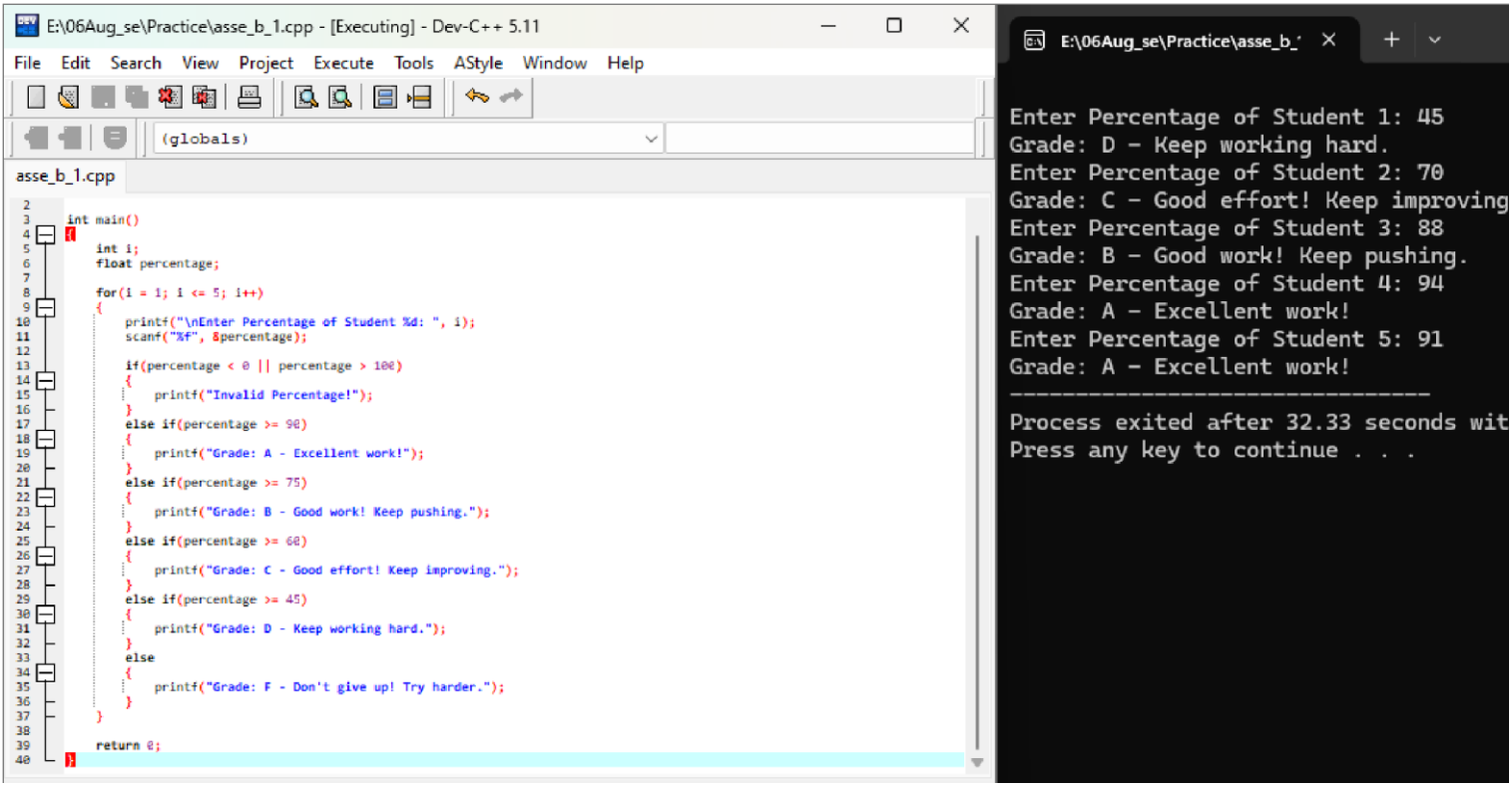
Assign a grade using if-else if: A (>= 90), B (>= 75), C (>= 60), D (>= 45), F (below 45).

Print the assigned grade and a one-line message for each band (e.g., 'B — Good work! Keep pushing.').

Handle invalid input: if the score is outside the range 0–100, print a clear error message and exit the program gracefully

Ans.

Enter Percentage of Student 1: 45 Grade:



D - Keep working hard.

Enter Percentage of Student 2: 70

Grade: C - Good effort! Keep improving.

Enter Percentage of Student 3: 88

Grade: B - Good work! Keep pushing.

Enter Percentage of Student 4: 94 Grade:

A - Excellent work!

Enter Percentage of Student 5: 91 Grade:

A - Excellent work!

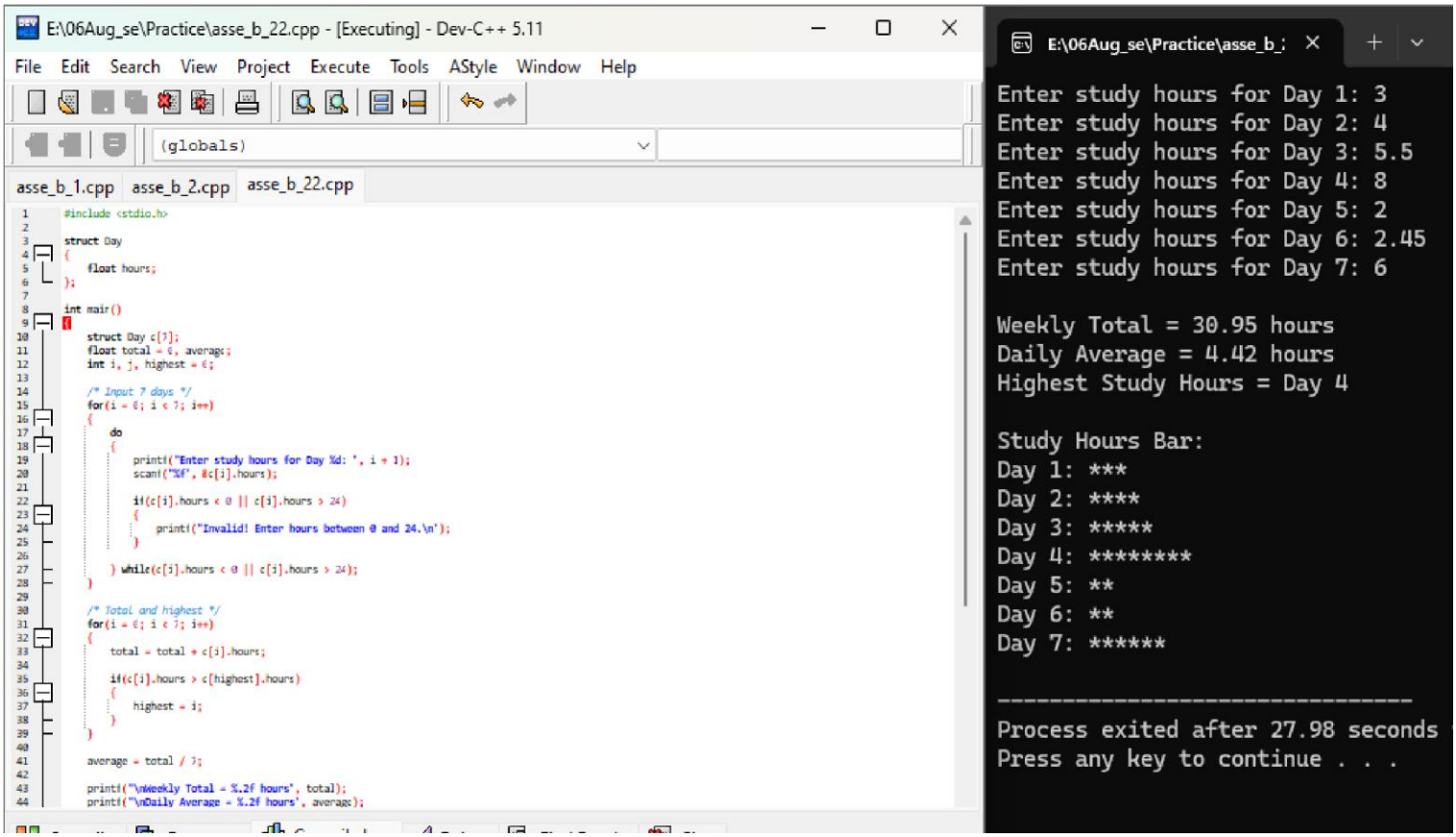
Q 2 Weekly Study Hours Analyser

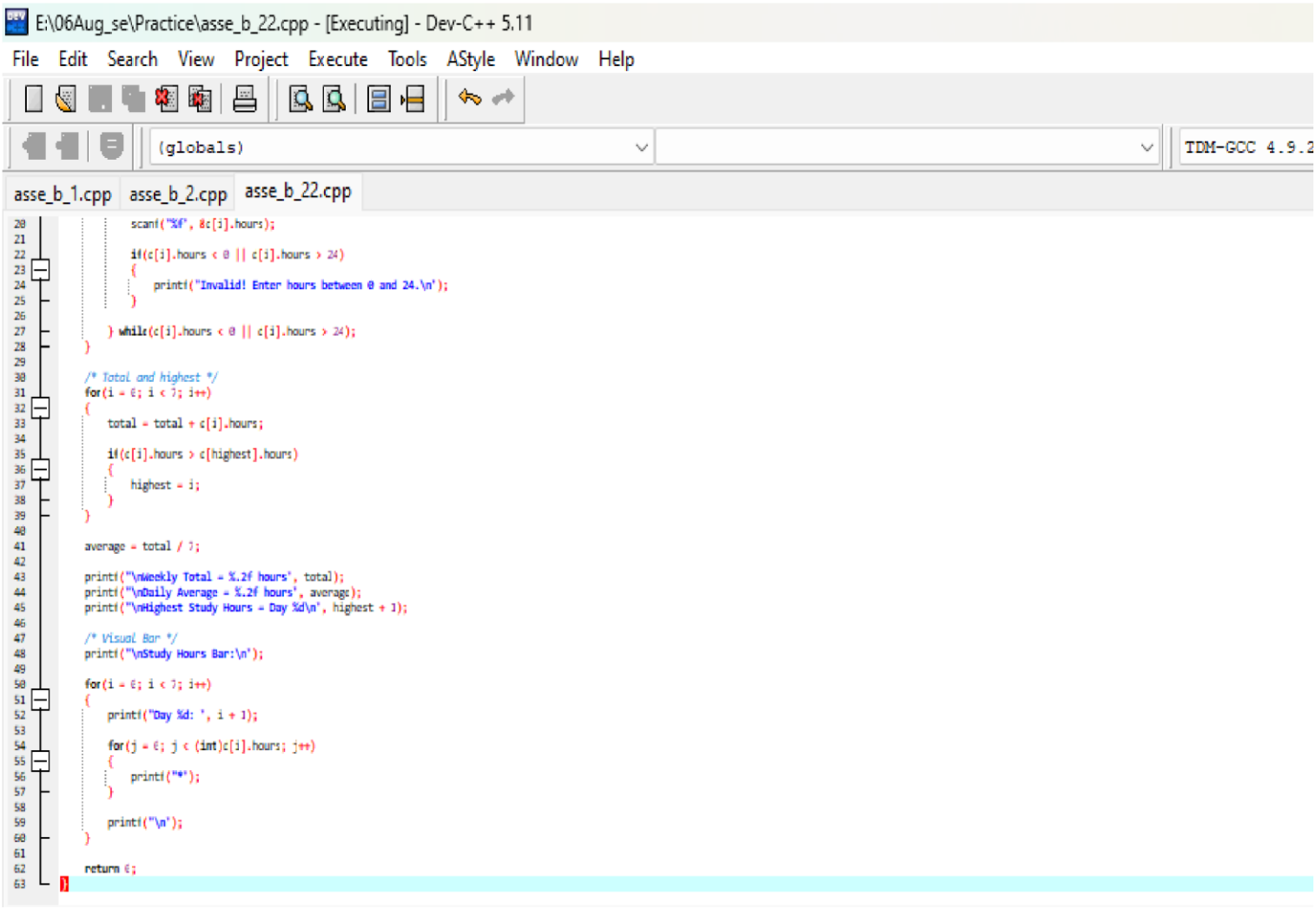
Use a for loop to accept 7 float values (one per day) and store them in a float array.

Calculate and print the weekly total, daily average, and the day number with the highest study hours.

Print a simple visual bar for each day: the day number followed by one asterisk (*) per hour studied, truncated to the nearest integer (e.g., Day 3: ***).

Reject and re-prompt for any day entry that is negative or greater than 24, before storing it in the array Ans.





Enter study hours for Day 1: 3

Enter study hours for Day 2: 4

Enter study hours for Day 3: 5.5

Enter study hours for Day 4: 8

Enter study hours for Day 5: 2

Enter study hours for Day 6: 2.45

Enter study hours for Day 7: 6

Weekly Total = 30.95 hours

Daily Average = 4.42 hours

Highest Study Hours = Day 4 Study Hours Bar:

Day 1: ***

Day 2: ****

Day 3: *****

Day 4: ********

Day 5: **

Day 6: **

Day 7: ******

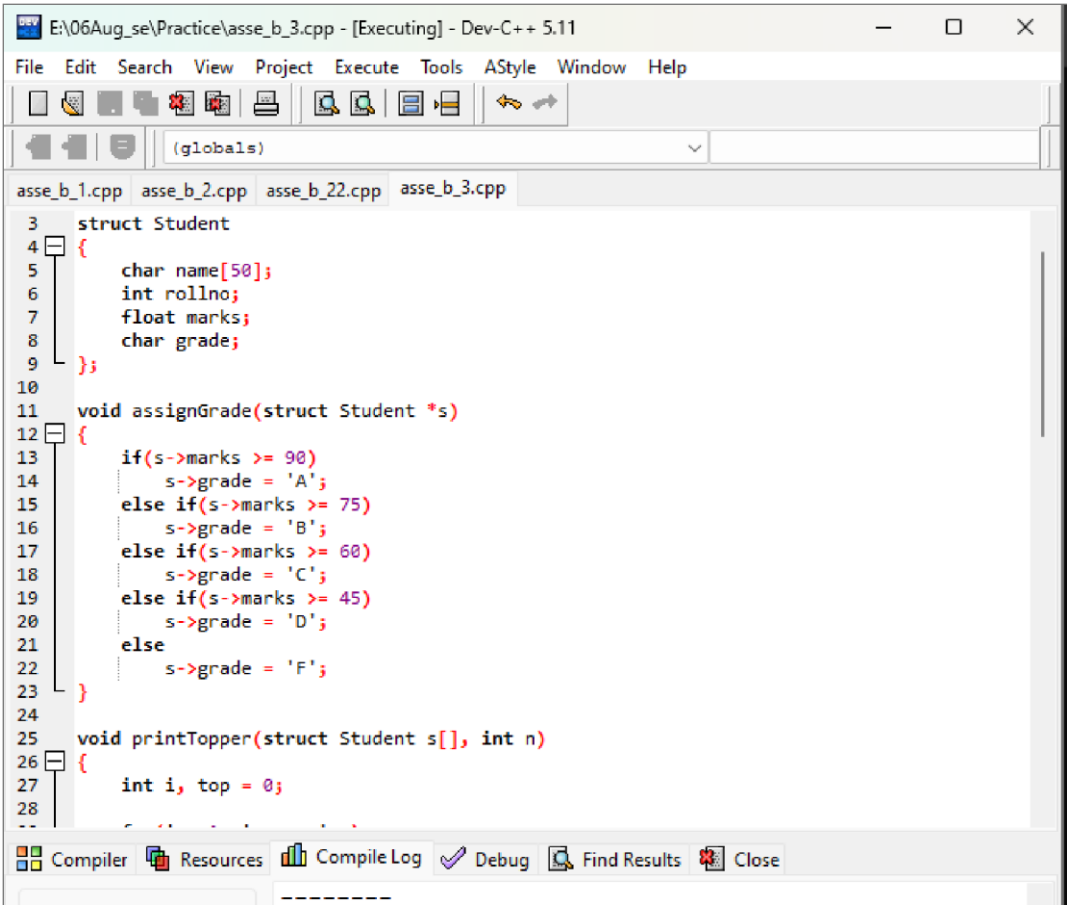
Q 3 Student Record Manager

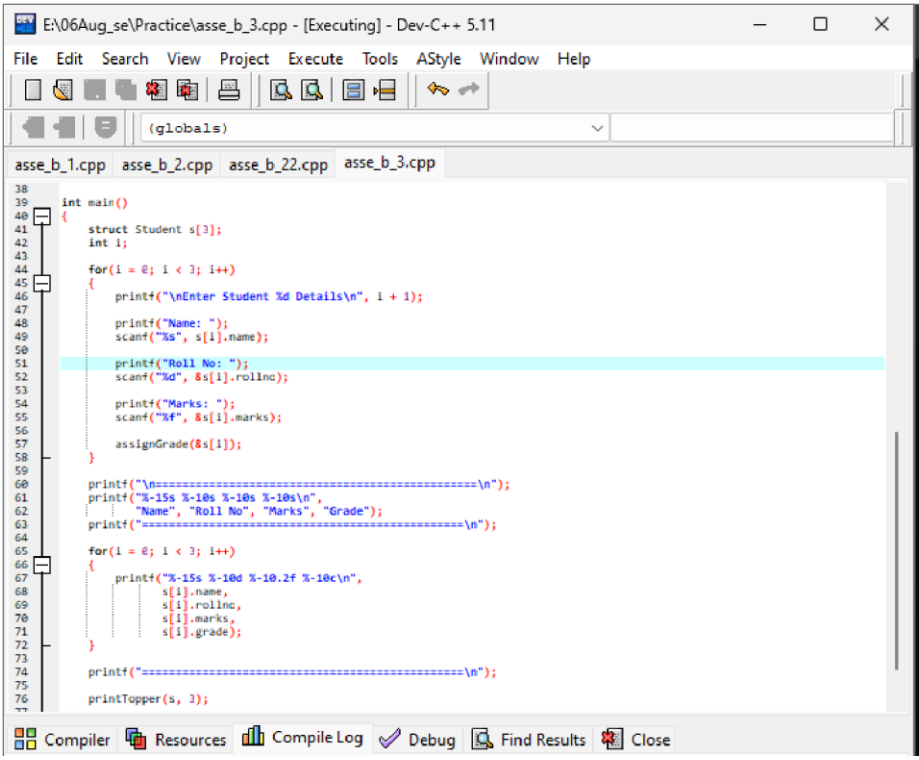
Define a struct Student with fields: name (char[50]), rollno (int), marks (float), and grade (char).

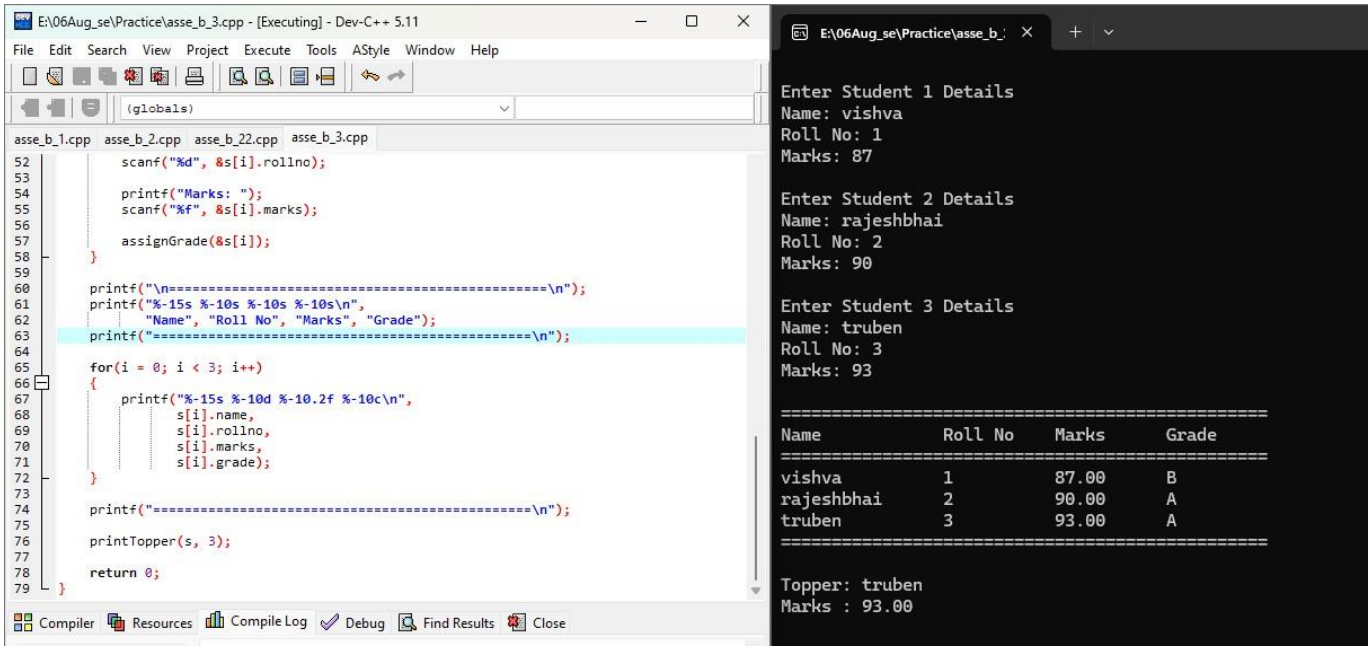
Write a function void assignGrade(struct Student *s) that sets the grade field based on the marks value using the same bands as Task 1. In main(), accept data for 3 students, call assignGrade() for each, and display all records in a formatted table with column headers.

Write a separate function void printTopper(struct Student[], int n) that prints the name and marks of the student with the highest marks.

Ans.


E:\06Aug_se\Practice\asse_b_3.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
asse_b_1.cpp asse_b_2.cpp asse_b_22.cpp asse_b_3.cpp
3 struct Student
4 {
5 char name[50];
6 int rollno;
7 float marks;
8 char grade;
9 };
10
11 void assignGrade(struct Student *s)
12 {
13 if(s->marks >= 90)
14 s->grade = 'A';
15 else if(s->marks >= 75)
16 s->grade = 'B';
17 else if(s->marks >= 60)
18 s->grade = 'C';
19 else if(s->marks >= 45)
20 s->grade = 'D';
21 else
22 s->grade = 'F';
23 }
24
25 void printTopper(struct Student s[], int n)
26 {
27 int i, top = 0;
28
Compiler Resources Compile Log Debug Find Results Close
E:\06Aug_se\Practice\asse_b_
Enter Student 1 Details
Name: vishva
Roll No: 1
Marks: 87
Enter Student 2 Details
Name: rajeshbhai
Roll No: 2
Marks: 90
Enter Student 3 Details
Name: truben
Roll No: 3
Marks: 93
Name Roll No Mark
vishva 1 87.0
rajeshbhai 2 90.0
truben 3 93.0
Topper: truben
Marks : 93.00


E:\06Aug_se\Practice\asse_b_3.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
asse_b_1.cpp asse_b_2.cpp asse_b_22.cpp asse_b_3.cpp
38
39 int main()
40 {
41 struct Student s[3];
42 int i;
43
44 for(i = 0; i < 3; i++)
45 {
46 printf("\nEnter Student %d Details\n", i + 1);
47
48 printf("Name: ");
49 scanf("%s", s[i].name);
50
51 printf("Roll No: ");
52 scanf("%d", &s[i].rollno);
53
54 printf("Marks: ");
55 scanf("%f", &s[i].marks);
56
57 assignGrade(&s[i]);
58 }
59
60 printf("\n===\n");
61 printf("%-15s %-10s %-10s %-10s\n",
62 "Name", "Roll No", "Marks", "Grade");
63 printf("===\n");
64
65 for(i = 0; i < 3; i++)
66 {
67 printf("%-15s %-10d %-10.2f %-10c\n",
68 s[i].name,
69 s[i].rollno,
70 s[i].marks,
71 s[i].grade);
72 }
73
74 printf("===\n");
75
76 printTopper(s, 3);
Compiler Resources Compile Log Debug Find Results Close
E:\06Aug_se\Practice\asse_b_
Enter Student 1 Details
Name: vishva
Roll No: 1
Marks: 87
Enter Student 2 Details
Name: rajeshbhai
Roll No: 2
Marks: 90
Enter Student 3 Details
Name: truben
Roll No: 3
Marks: 93
Name Roll No Marks Grade
vishva 1 87.00 B
rajeshbhai 2 90.00 A
truben 3 93.00 A
Topper: truben
Marks : 93.00



Q.4 Personal Expense Logger

Define a struct Expense with fields: category (char[30]) and amount (float). Allocate an array to store up to 10 Expense entries.

Present a menu with 3 options: (1) Add Expense, (2) View All Expenses, (3) Save & Exit. Loop until the user selects option 3.

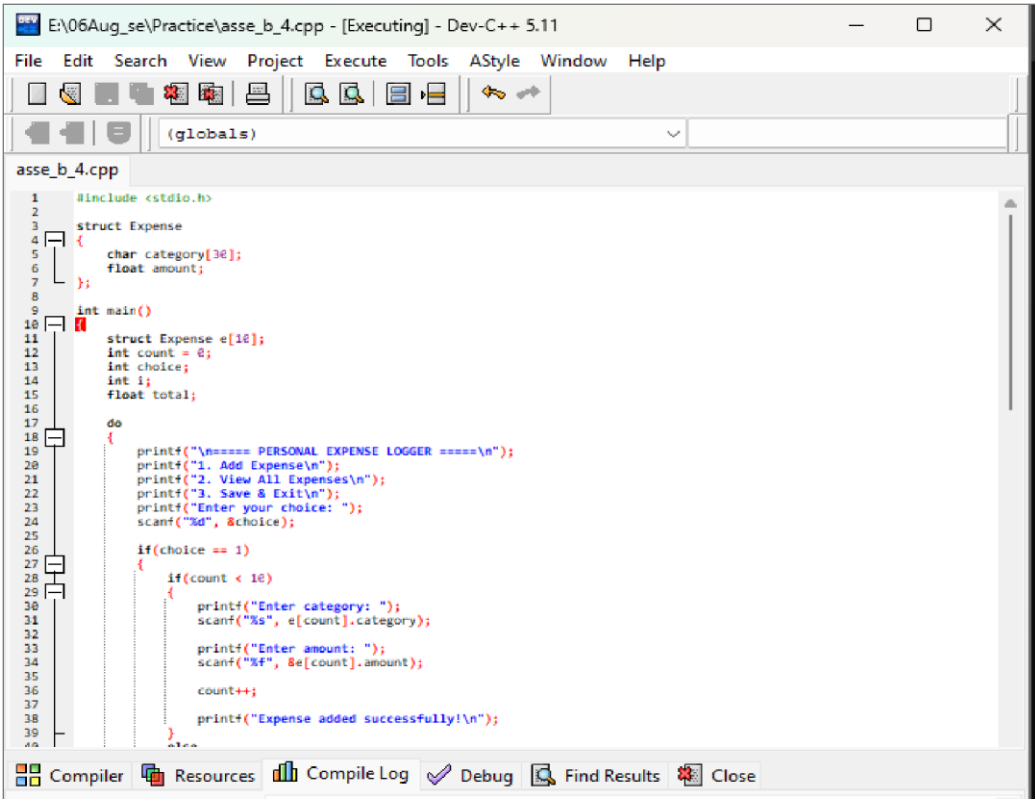
In the View option, display all logged expenses in a formatted list and print the running total at the bottom.

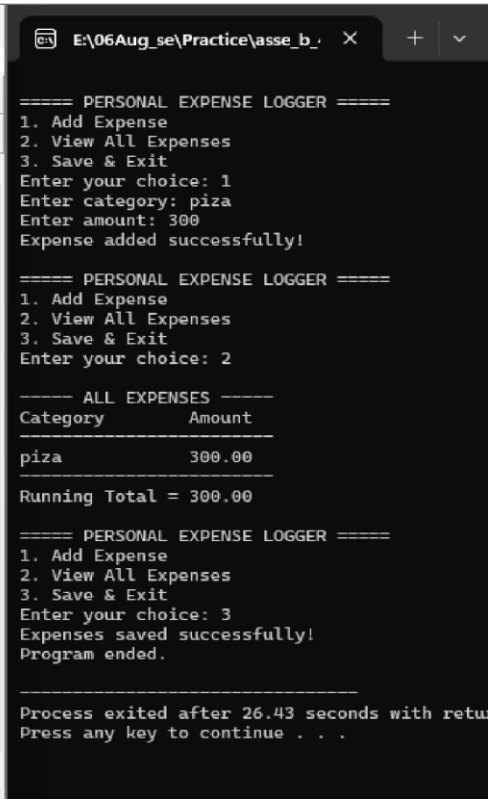
On exit (option 3), write all expense records to a file named expenses.txt using fprintf(), with one record per line in the format: category,amount.

Ans.

```
#include <stdio.h>

struct Expense
{
```






```
    char category[30];
float amount;
};

int main()
{
    struct Expense e[10];
int count = 0;     int
choice;     int i;     float
total;

    do
    {
 printf("\n===== PERSONAL EXPENSE LOGGER
\n");
printf("1. Add Expense\n");
printf("2. View All Expenses\n");
printf("3. Save & Exit\n");
```

```
printf("Enter your choice: ");
scanf("%d", &choice);

        if(choice == 1)
        {
if(count < 10)
            {
                printf("Enter category: ");
scanf("%s", e[count].category);

                printf("Enter amount: ");
scanf("%f", &e[count].amount);

count++;

                printf("Expense added successfully!\n");
            }
else
            {
                printf("Maximum 10 expenses allowed!\n");
            }
        }

        else if(choice == 2)
        {
total = 0;
```

```
            printf("\n----- ALL EXPENSES -----\n");
printf("Category\tAmount\n");            printf("-----------------------
\n");
             for(i = 0; i < count;
i++)
            {
printf("%s\t\t%.2f\n",
e[i].category,
e[i].amount);

                total = total + e[i].amount;
            }
             printf("------------------------\n");
printf("Running Total = %.2f\n", total);
        }

        else if(choice == 3)
        {
            FILE *fp;

            fp = fopen("expenses.txt", "w");
            if(fp == NULL)
            {
                printf("File could not be created!\n");
            }            else            {
for(i = 0; i < count; i++)
                {                    fprintf(fp,
"%s,%.2f\n",
```

```
e[i].category,
e[i].amount);
                }

fclose(fp);

                printf("Expenses saved successfully!\n");
printf("Program ended.\n");
            }
        }
else
        {               printf("Invalid
choice!\n");
        }

    } while(choice != 3);

    return 0;
}
```

C .C — Mini Capstone Project

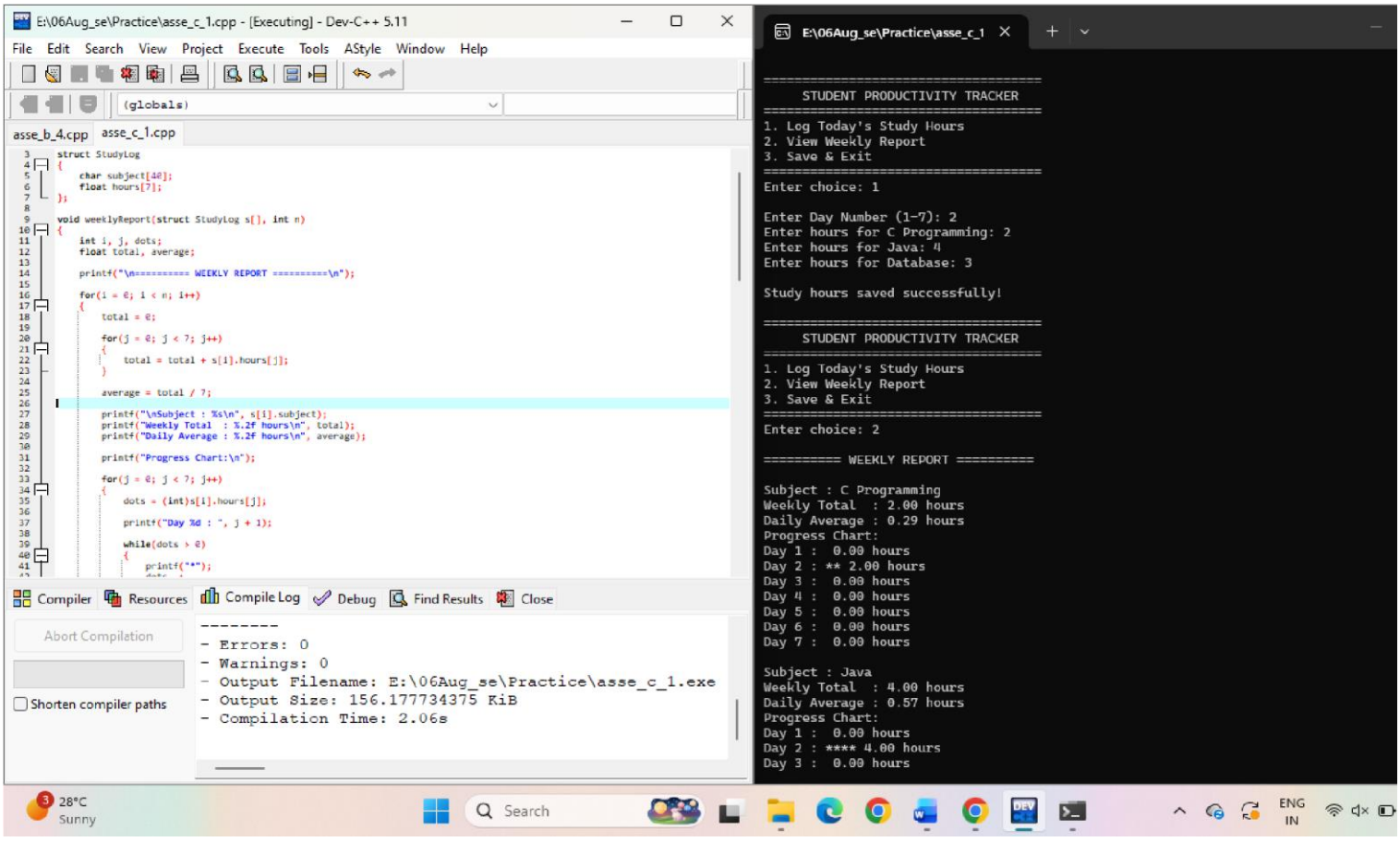
Q.1 The program must be menu-driven with at least 3 options: (1) Log Today's Study Hours, (2) View Weekly Report, (3) Save & Exit.

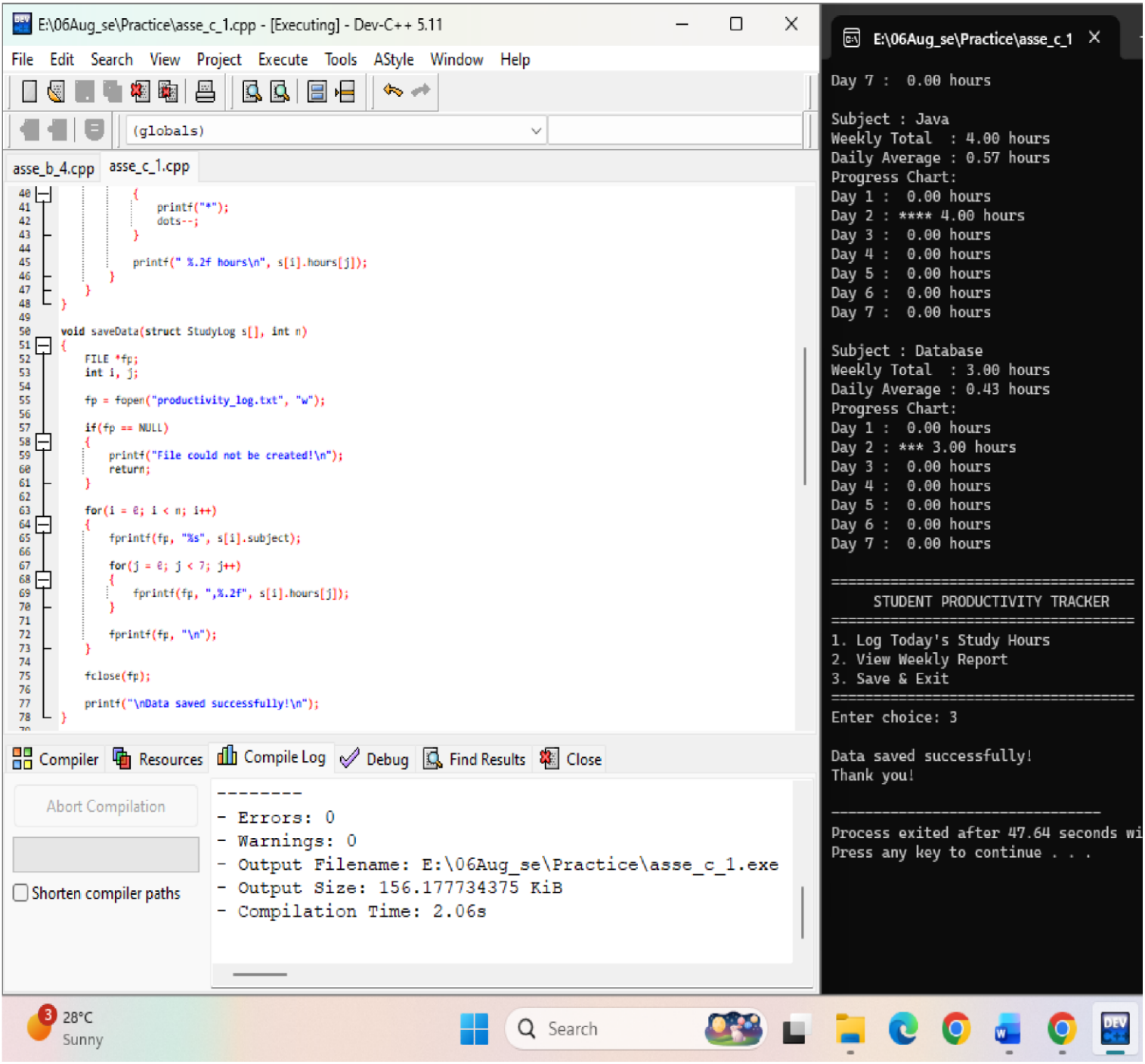
Define a struct StudyLog { char subject[40]; float hours[7]; } and create an array of at least 3 subject records.

Write a function that calculates and displays the weekly total hours and daily average for each subject.

Display a simple text-based progress chart: for each subject, print one filled dot (•) per hour studied that day (truncate to nearest integer).

On exit, save all records to a file named productivity_log.txt using fprintf(), with each subject's name and 7 daily values written as a single comma-separated line.






```
#include <stdio.h>

struct StudyLog
{
    char subject[40];
float hours[7];
};
```

```
void weeklyReport(struct StudyLog s[], int n)
{     int i, j, dots;
float total, average;

    printf("\n========== WEEKLY REPORT ==========\n");

    for(i = 0; i < n; i++)
    {         total
= 0;
         for(j = 0; j < 7;
j++)
        {             total = total +
s[i].hours[j];
        }

        average = total / 7;

        printf("\nSubject : %s\n", s[i].subject);
printf("Weekly Total  : %.2f hours\n", total);
printf("Daily Average : %.2f hours\n", average);

        printf("Progress Chart:\n");
         for(j = 0; j < 7;
j++)
        {             dots =
(int)s[i].hours[j];
printf("Day %d : ", j + 1);
```

```
            while(dots > 0)
            {
printf("*");
dots--;
            }

            printf(" %.2f hours\n", s[i].hours[j]);
        }
    }
}

void saveData(struct StudyLog s[], int n)
{
    FILE *fp;
    int i, j;

    fp = fopen("productivity_log.txt", "w");

    if(fp == NULL)
    {
        printf("File could not be created!\n");
return;
    }

    for(i = 0; i < n; i++)
    {         fprintf(fp, "%s",
s[i].subject);
```

```
        for(j = 0; j < 7;
j++)
        {           fprintf(fp, ",%.2f",
s[i].hours[j]);
        }
fprintf(fp, "\n");
    }

    fclose(fp);

    printf("\nData saved successfully!\n");
}

int main()
{
    struct StudyLog s[3] =
    {
        {"C Programming", {0,0,0,0,0,0,0}},
        {"Java", {0,0,0,0,0,0,0}},
        {"Database", {0,0,0,0,0,0,0}}
    };

    int choice, day, i;

    do
    {
```

```
printf("\n=====================================\n");
printf("     STUDENT PRODUCTIVITY TRACKER\n");

printf("=====================================\n");
        printf("1. Log Today's Study Hours\n");
printf("2. View Weekly Report\n");         printf("3.
Save & Exit\n");

printf("=====================================\n");


        printf("Enter choice: ");
scanf("%d", &choice);

        if(choice == 1)
        {
            printf("\nEnter Day Number (1-7): ");
scanf("%d", &day);

            if(day >= 1 && day <= 7)
            {                   for(i = 0; i
< 3; i++)
                {
                    printf("Enter hours for %s: ", s[i].subject);
scanf("%f", &s[i].hours[day - 1]);
                }

                printf("\nStudy hours saved successfully!\n");
            }
else
```

```
            {                printf("Invalid
day!\n");
            }
        }
        else if(choice == 2)
        {
            weeklyReport(s, 3);
        }
        else if(choice == 3)
        {
            saveData(s, 3);
printf("Thank you!\n");
        }
else
        {            printf("Invalid
choice!\n");
        }

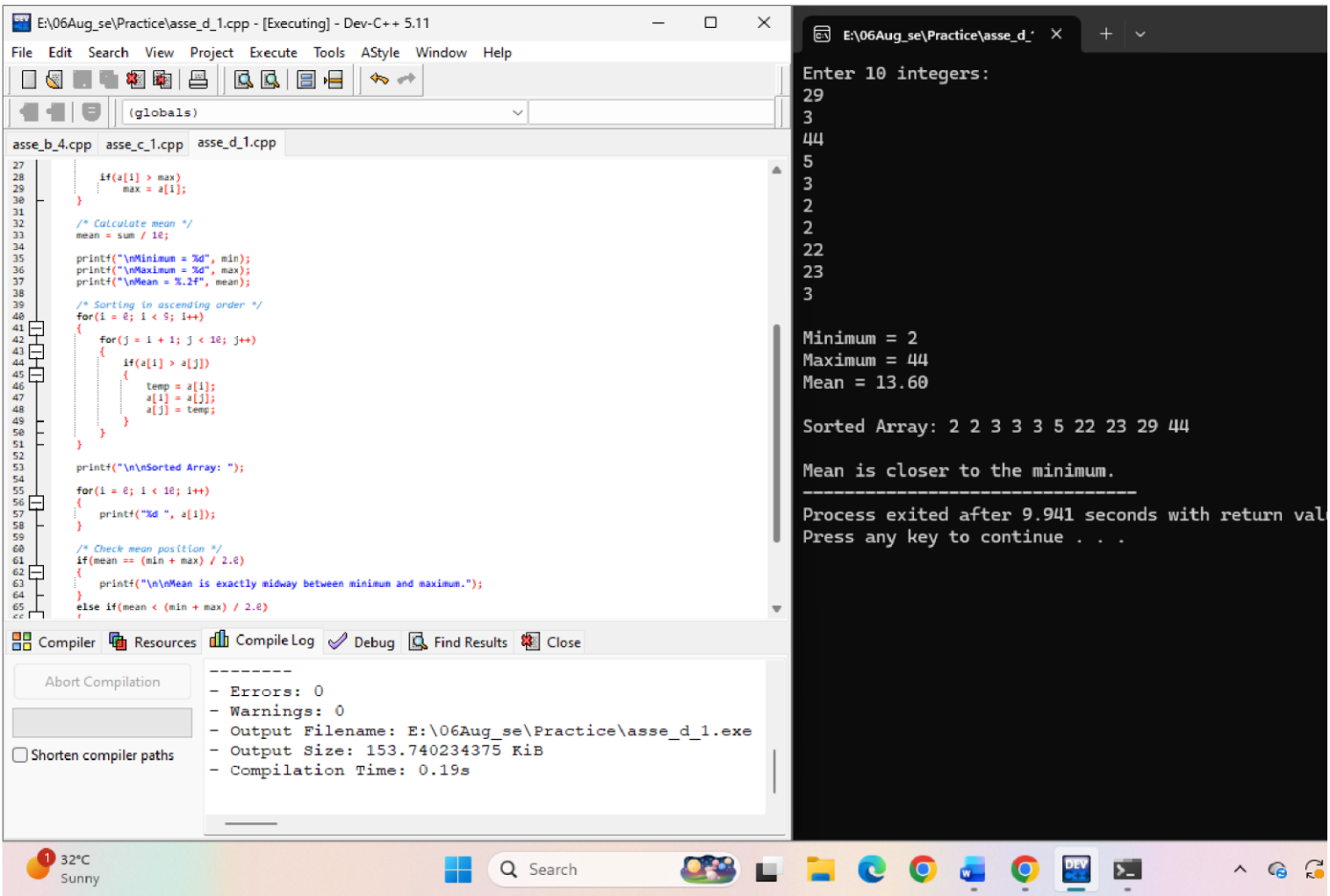
    } while(choice != 3);

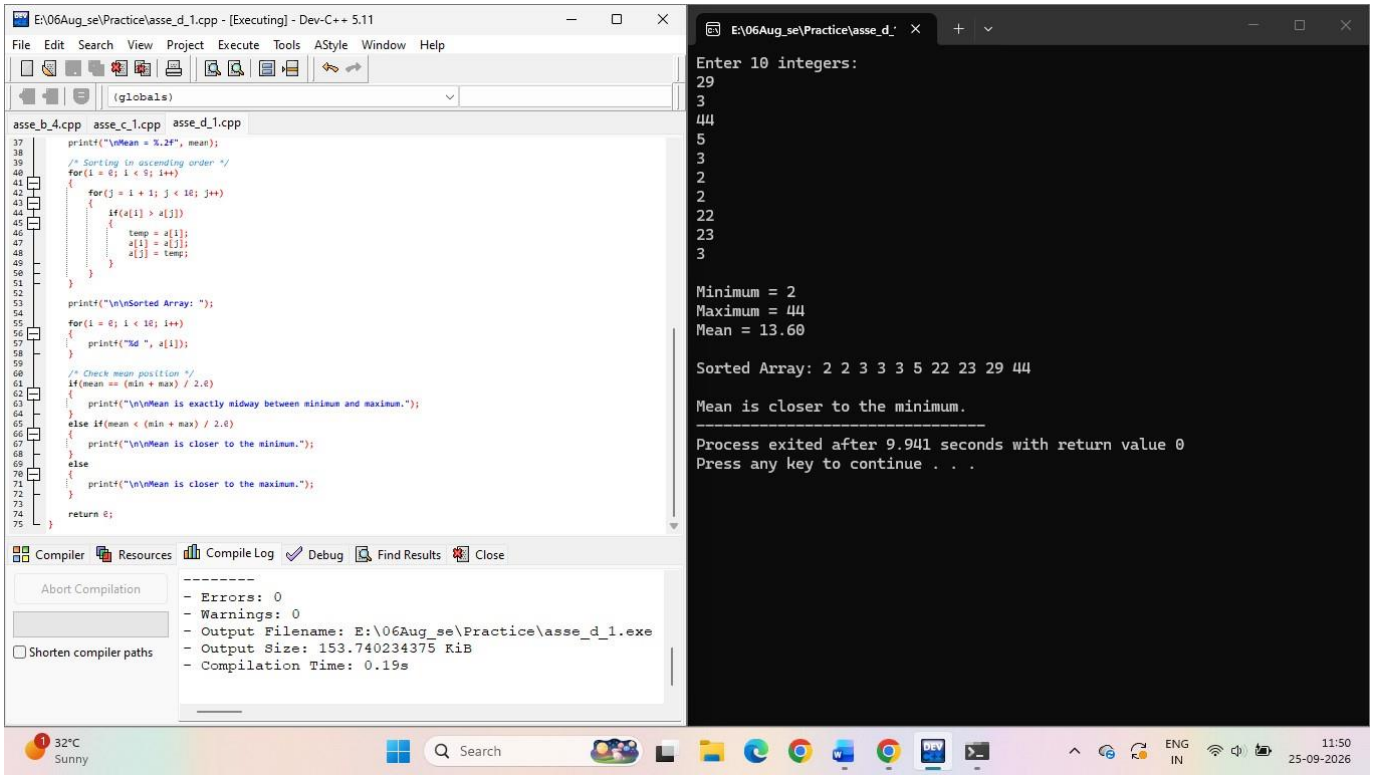
    return 0;
}
```

D .D — AI-Augmented Learning

Q.1 Accepts exactly 10 integers from the user using a loop and stores them in an array. Finds and displays the maximum value, minimum value, and arithmetic mean (displayed as a float with 2 decimal places). Sorts the array in ascending order using any sorting method and displays the sorted list. Prints whether the mean is closer to the minimum, closer to the maximum, or exactly midway between them.

Ans.






```
#include <stdio.h>

int main()
{
    int a[10];     int i, j,
temp;     int min, max;
float sum = 0, mean;

    printf("Enter 10 integers:\n");

    /* Input using loop */
for(i = 0; i < 10; i++) {

        Sca0nf("%d", &a[i]);
sum = sum + a[i];
    }
```

```
    /* Find minimum and maximum */
min = a[0];     max = a[0];

    for(i = 1; i < 10; i++)
    {         if(a[i] <
min)              min
= a[i];

         if(a[i] > max)
max = a[i];
    }

    /* Calculate mean */
mean = sum / 10;

    printf("\nMinimum = %d", min);
printf("\nMaximum = %d", max);     printf("\nMean
= %.2f", mean);

    /* Sorting in ascending order */     for(i = 0; i < 9; i++)
    {
        for(j = i + 1; j < 10; j++)
        {
            if(a[i] > a[j])
```

```
            {
                temp = a[i];
a[i] = a[j];                a[j]
= temp;
            }
        }
    }

    printf("\n\nSorted Array: ");

    for(i = 0; i < 10; i++)
    {
        printf("%d ", a[i]);
    }

    /* Check mean position */
if(mean == (min + max) / 2.0)
    {
        printf("\n\nMean is exactly midway between minimum and
maximum.");
    }
    else if(mean < (min + max) / 2.0)
    {
        printf("\n\nMean is closer to the minimum.");
    }
else
```

```
    {
        printf("\n\nMean is closer to the maximum.");
    }

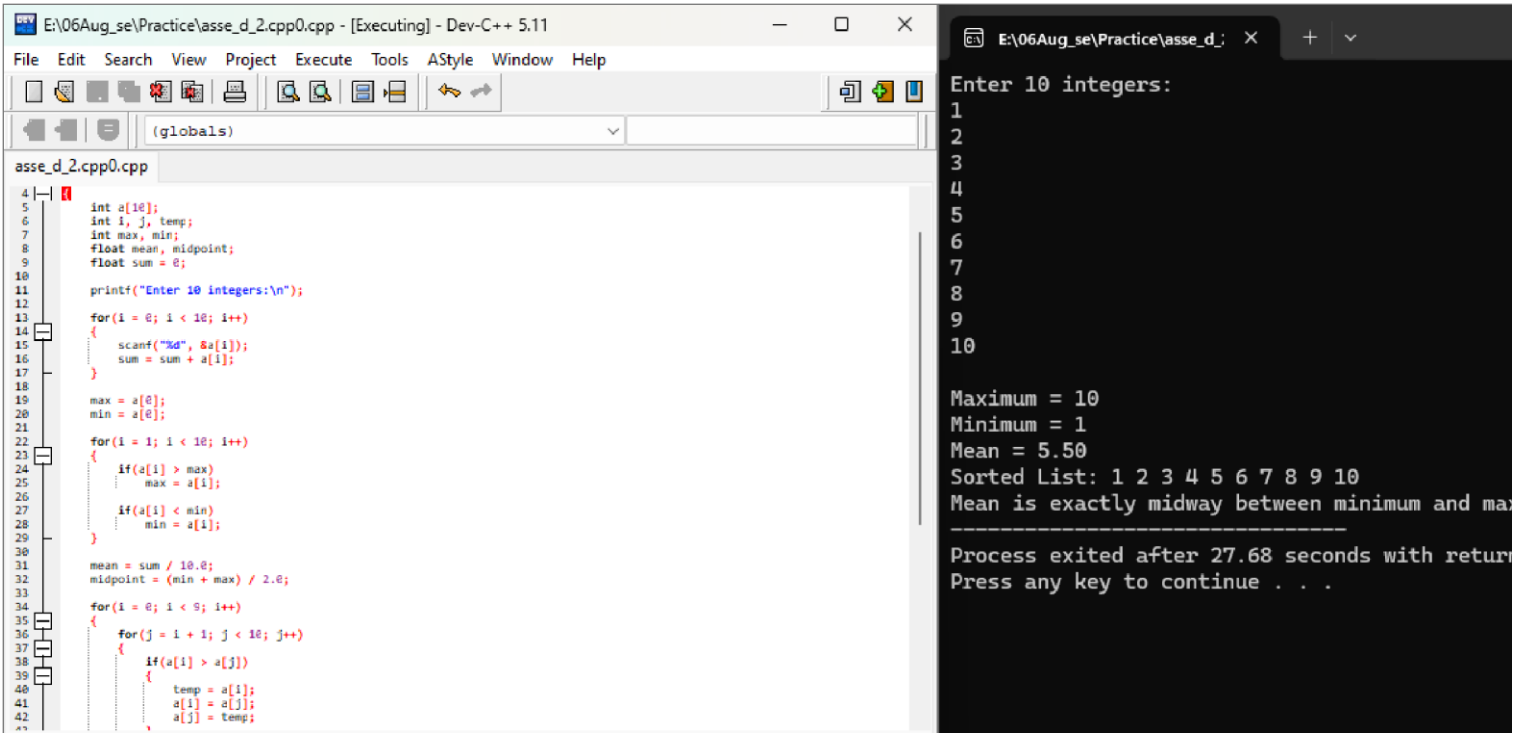

    return 0;
}
```

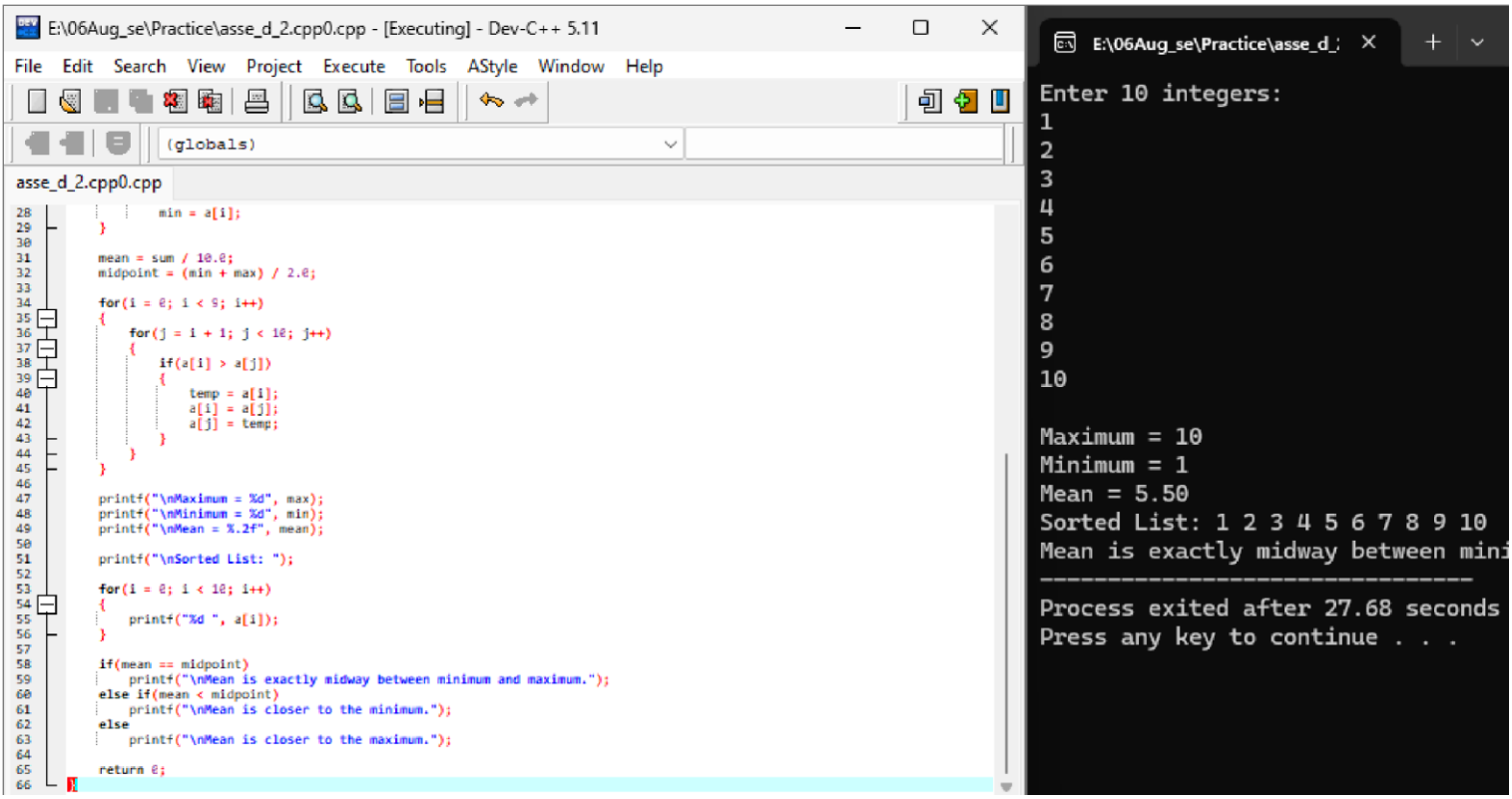
Q 2. TEST & DEBUG (WITHOUT AI)

SUBMIT All 3 items required 1 The exact prompt(s) you gave the AI tool.

2 The AI's original code and your corrected version (submit both as separate .c files).

3 A 3–4 line note explaining what you changed and why the AI's original version needed the correction





#include <stdio.h>

int main()

```
{     int
a[10]; int i,
j, temp;

    int max, min;     float
mean, midpoint;     float
sum = 0;

    printf("Enter 10 integers:\n");

    for(i = 0; i < 10; i++)
    {
        scanf("%d", &a[i]);
sum = sum + a[i];
    }

    max = a[0];
min = a[0];

    for(i = 1; i < 10; i++)
    {         if(a[i] >
max)             max
= a[i];
         if(a[i] <
min)             min
= a[i];
```

```
    }

    mean = sum / 10.0;
    midpoint = (min + max) / 2.0;

    for(i = 0; i < 9; i++)
    {
        for(j = i + 1; j < 10; j++)
        {             if(a[i]
> a[j])
            {
temp = a[i];
a[i] = a[j];
a[j] = temp;
            }
        }
    }

    printf("\nMaximum = %d", max);
printf("\nMinimum = %d", min);     printf("\nMean
= %.2f", mean);

    printf("\nSorted List: ");

    for(i = 0; i < 10; i++)
    {
```

```
        printf("%d ", a[i]);
    }

    if(mean == midpoint)
        printf("\nMean is exactly midway between minimum and
maximum.");
    else if(mean < midpoint)
        printf("\nMean is closer to the minimum.");
else
        printf("\nMean is closer to the maximum.");

    return 0;
}
```

The program accepts exactly 10 integers and stores them in an array.
It finds the maximum, minimum and arithmetic mean of the numbers.
It sorts the array in ascending order and displays the sorted list.
It also checks whether the mean is closer to the minimum, maximum, or exactly midway.